

BME 313L: Introduction to Numerical Methods in Biomedical EngineeringHonor Code: *I have neither given nor received aid on this quiz.*

UT EID: _____

Printed Name: _____

Signature: _____

Problem 1 (10 Points): Recall the Babylonian method for computing the $\sqrt{S}$:

$$x_{k+1} = \frac{\frac{S}{x_k} + x_k}{2}.$$

```

01 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
02 function x = babyRoot(x0,S)
03 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
04 % Input:
05 %   x0: is the initial guess
06 %   S: we wish to compute sqrt(S)
07 %
08 % Output:
09 %   x = sqrt(S);
10
11 % Define the stopping tolerance
12 tol = 10^-6;
13
14 % Conduct the iteration
15 for k = 1:100
16
17     % Compute the new iterate
18     x = ( S/x0 + x0 )/2;
19
20     % Compute the approximate relative error
21     E = abs( x-x0 )/abs(x);
22
23     % The new iterate is now our current one
24     x0 = x;
25
26     % Check stopping criteria
27     if(E < tol )
28         break; % If true, then the for-loop with stop
29     end
30 end

```

The code babyRoot attempts to implement the method in MATLAB. However, this happens on the first test run:

```
>> x = babyRoot(0,4)
```

```
x =
```

```
    Inf
```

Determine the source of the error and describe how to correct it. If the code has a bug, list the line numbers and the correct code.

ANSWER (10 pts): There is no bug in the code. The failure is caused by the initial guess $x_0 = 0$ that was passed in the call `babyRoot(0,4)`.

Line 18 evaluates $S/x_0 = 4/0$, which is Inf, so x becomes Inf on the first pass. Line 21 then computes $E = \text{abs}(\text{Inf} - 0)/\text{abs}(\text{Inf})$, which is NaN, and the test $\text{NaN} < \text{tol}$ on line 27 is false, so the loop never breaks. Line 24 sets $x_0 = \text{Inf}$, every later iterate is Inf, and the function runs all 100 iterations and returns Inf.

Correction: Call the function with a nonzero guess, for example `babyRoot(1,4)` or `babyRoot(S,S)`, or guard the input at the top of the function:

```

    if( x0 == 0 )
        x0 = S;
    end

```

September 11, 2026

BME 313L Quiz 2 - ANSWERS

Problem 2 (4 Points): I am working on a new program that stores numbers as an 8-bit signed integer, but I want to make sure that the data type provides a sufficient range of values. What is the range of values that can be represented by a signed 8-bit integer.

ANSWER (4 pts): -128 to 127.

One of the eight bits carries the sign, leaving seven bits of magnitude. "Negative 0" does not mean anything mathematically, so by convention the negative side reaches one step further from zero than the positive side and the range is -2^7 to $2^7 - 1$

Problem 3 (10 Points): The function below computes the approximate relative error for each entry of a vector of approximations, a , against the corresponding vector of true values t .

```
01 function E = relErr(a,t)
02 % a: vector of approximations
03 % t: vector of true values
04 n = length(a);
05 E = zeros(1,n);
06 for k = 1:n
07     E(k) = abs( a(k) - t(k) )/abs( t(k) );
08 end
```

A) The line 5 preallocates E before the loop begins. If that line were deleted, would `relErr` still return the correct result? Explain what changes.

ANSWER A) (3 pts): Yes, it still returns the correct numbers.

Without line 5, E does not exist when the loop starts, so MATLAB creates it on the first assignment and grows it by one element on every pass. Each growth allocates a new, larger array and copies everything written so far into it, so the work grows with the square of n rather than linearly with n . The answer is unchanged, the function is simply much slower for large vectors.

B) Re-write the body of `relErr` so that E is computed with a single vectorized expression and no for-loop:

ANSWER B) (5 pts):

```
E = abs(a-t)./abs(t);
```

The division must be element-wise. A plain `/` is matrix right division, which returns a single scalar rather than a vector of errors

C) Write a single line of MATLAB that stores the values 0.00, 0.01, 0.02, ..., 0.50 in a row vector X , so that $X(1) = 0.00$ and $X(51) = 0.50$:

ANSWER C) (2 pts):

```
X = 0:0.01:0.5;
```

The vector has 51 entries, from $X(1) = 0.00$ through $X(51) = 0.50$.

Problem 4 (3 points) Both of the expressions below are zero in exact arithmetic. Explain why MATLAB returns exactly zero for the first and a nonzero value for the second.

```
>> (1.0 + 2.0) - 3.0
ans =
    0
>> (0.1 + 0.2) - 0.3
ans =
 5.5511e-17
```

ANSWER (3 pts): The difference is whether the operands themselves can be stored exactly.

1.0, 2.0 and 3.0 are small integers, so each is represented exactly in binary, and $1.0 + 2.0$ is exactly 3.0. Nothing is rounded anywhere in the first expression, so the subtraction returns exactly zero.

0.1, 0.2 and 0.3 are infinite repeating fractions in binary, so each is rounded to the nearest double as it is entered. The stored value of $0.1 + 0.2$ is slightly larger than the stored value of 0.3, and the subtraction returns the difference between those two roundings, about $5.55e-17$. That residue is on the order of machine epsilon, $2.2e-16$, scaled to numbers near 0.3.

Problem 5 (3 points) What will be the output of the following code? Explain the reasoning for your answer.

```
01 x = 0;
02 for k = 1:10
03     x = x + 0.1;
04 end
05
06 if( x == 1 )
07     disp('Done')
08 else
09     disp('Nope')
10 end
```

ANSWER (3 pts): The output is

Nope

0.1 has no exact binary representation, so it is rounded on entry and the result of each of the ten additions is rounded again. Those errors do not cancel. After the loop, x holds 0.9999999999999999, not 1, so the test $x == 1$ on line 06 is false and the else branch runs. Note that format short displays this value as 1.0000, which is not what $==$ compares.

The correct test is a tolerance comparison:

```
if( abs(x - 1) < tol )
```